Task 1.1P

Question 1

A mid-sized SaaS company frequently releases new features to stay competitive. However, security measures—such as input validation, strong encryption ciphers, and others—are only added after incidents of exploitation or suspicious activities are reported.

Explain which security principle is being violated and how this can be mitigated. Additionally, discuss how the organization can remain competitive while ensuring their software is secure.

The security by design principle is being violated here. The system's security is being treated more like a reactive addition rather than a proactive core requirement of the systems' design. Security requirements like input validation, encryption, access control and secure error handling should be identified and planned using the requirement analysis stage and should be assessed through threat modeling during the design stage, and should finally be implemented during the implementation/development stage. At the very end, they should also go through thorough testing.

The company can remain being competitive by making security a part of the normal development process. Security related tasks should be included in the project plan and every feature should meet the basic security requirements before its considered as "complete". Features like automated SAST/DAST, dependency scanning, security tests and other security tooling/approaches can be integrated into the development workflow / CI/CD pipeline. This will allow the security checks to occur during every release without slowing down the speed of the delivery too much. Detecting vulnerabilities early will also save the company from potential cyber attacks in the feature.

Question 2

An enterprise application relies on an authentication mechanism that uses relational database management systems (RDBMS) to verify user credentials. During an unexpected high-usage situation, the RDBMS fails. The developers used a fallback mechanism which had error and also bypassed credentials checks. As a result, even unauthenticated requests are granted access to the application. Explain briefly which software security principle was violated and describe the process to mitigate it.

The app has failed to fail securely. The fallback system currently fails open – meaning, when the authentication process cannot be completed, the system will just allow access. The correct approach should be to treat the user as unauthenticated and deny access.

When the authentication database is not available, the application should return a response thats a controlled error with something like "service unavailable". If it's a web app, maybe return it with a 5XX response code. The fallback process must NEVER skip the actual verification of credentials and the authentication. Developers and the DBAs should also take proper care to ensure the database is running properly and redundantly.

Threat tree guidance also recommends testing what happens when the database becomes fully unavailable to make sure that this application handles that safely as well. Maybe the RDBMS goes down or takes too long to respond and times out. The application should be able to handle that.

Question 3

A mature DevSecOps team at an organisation enforces a policy that prohibits developers approving their own code for release. However, due to staffing constraints, a developer who implemented a major feature occasionally participates in QA testing and also handles product release deadlines. Over time, this becomes routine practice within the team. Explain which secure software development principles are being followed or violated in this situation?

They are properly following the separation of duties when it comes to the approval of code and pull requests. However, letting the same developer implement, test and release the feature violates the principle being followed. A vulnerability, authorized changes or just an insecure implementation could pass through every stage without being detected.

Development, QA approval and production release decisions must be separate responsibilities. If the organization has a limited number of staff, they could perform cross team reviews, and maybe also implement review by a second reviewer before approval of the release. Automated CI/CD security checks may support this process, but they must not replace reviews done by humans.

Question 4

A weather forecasting platform allows third-party services to access their forecasting pplication through a public API. The platform also communicates with other internal microservices and APIs. However, the software provider secures only the public APIs, assuming that the internal microservices and APIs are safe because they operate within a trusted network. Which secure coding principle should developers apply to minimize the security risks associated with unsecured internal services and APIs? Explain briefly.

They should do complete mediation. Every request must be authenticated and authorized – even when it comes it from the internal network. Nothing should be trusted, including internal network services just because of their location in the network.

Each micro service should use its own authenticated identity, with least privilege authorization, input validation and also encrypted communication (like using TLS on both ends). The network should be properly segmented and logging should be enabled for network related activity. Threat modeling shows that trust boundaries can exist inside a firewall and between internal services – this basically means that internal APIs (even though they are “internal”) are stall a part of the applications attack surface.

Question 5

The mitigation strategy for CSRF attacks requires applying security measures at various levels such as input validation, validating session cookies, verifying origin headers. What secure coding principle is being applied in this case? Explain briefly.

This is the defense in depth principle. It basically means that the application uses multiple security control approaches/mechanisms together. If one approach fails or gets bypassed, the other one will still be there without immediately compromising the system.

The CSRF protection, session cookie validation, SameSite, Secure and HttpOnly cookie attributed, anti CSRF tokens, origin header checks and proper input validation will protect against CSRF attacks at different levels in different ways. This means that the attacker would need to bypass all of these security layers instead of just exploiting one single vulnerability.

Question 6

An “Application A” makes an API call to email handling “Application B” to generate a to-do list from user flagged emails. However, the developer also grants read access to the contact list which is not required by the “Application A”. Which secure coding principle should developers apply to reduce the security risks posed associated with granting unnecessary access rights? Explain briefly.

The principle of least privilege is being violated and should be followed here. It basically means to give permissions to only whats absolutely necessary. Application A only needs access to the emails flagged by the user so it can proceed to continue the todo list. Giving it access to the contacts list is absolutely unnecessary and will introduce a large attack surface unnecessarily. If the API token is compromised, an attacker would be able to enumerate all of those contacts and compromise them.

The contacts list permission should therefore be removed. The API token used in the application should only require the absolute minimum scopes for processing the selected emails. Application B should also perform authorization checks on every API endpoint instead of just simply relying on the permissions requested by Application A.

Question 7

An organization enforces a policy that provides the administrator with only one account:-an administrative account. This account is used to perform both administrative tasks and routine, day-to-day activities by the administrator. Explain which security principle is being violated in this scenario and how this can be mitigated.

This again is a violation of the principle of least privilege. Using the administrator account with elevated permissions for every day tasks even when a lot of those permissions are not really being used is bad. If a malicious threat actor gets access to this account either via phishing/BEC or a stealer, they will get full administrative access to the systems and can cause so much more damage.

I’d make the administrator should have two separate accounts. One standard user account for normal day to day activities and an administrative account to be used only when that elevated permissions are being dealt with. The administrator account must require MFA, a good password policy, and possibly even the use of physical security keys like YubiKey for the best security. All actions performed by this admin user account should also be logged for better security, auditing, and even Tools like sudo or linux PAM can elevate permissions temporarily without giving the user any of those elevated permissions permanently.

—-

I’m have years of experience in web development and with AWS. All of my answers are slightly influenced from my personal knowledge and experience too.